

Entrepôt de Données de Santé — CHU

Module Big Data · M2 · Épreuve E05

Architecture, traitements, indicateurs, automatisation. L'analyse par indicateur et les captures des dashboards figurent dans l'annexe détaillée.

1. Le besoin

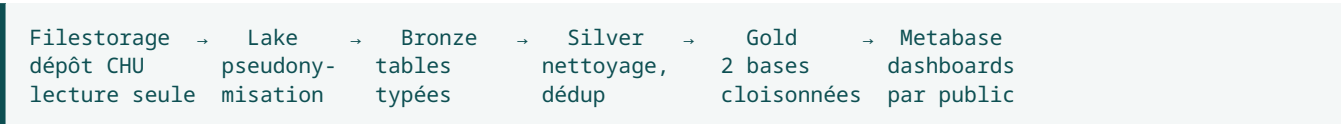
Le CHU exporte chaque jour des données éparpillées entre dossier patient, urgences et monitoring, dans des formats hétérogènes (CSV, JSON imbriqué, Parquet). Deux publics doivent en tirer parti sans voir les mêmes données : le **pilotage** (direction, cadres — occupation, flux, qualité des soins, intensité du plateau technique) et la **recherche clinique** (épidémiologistes — cohortes, distributions, comorbidités).

Les fichiers sources contiennent l'identité réelle des patients. Les données de santé relevant de l'article 9 du RGPD, la conformité **conditionne** l'architecture au lieu de s'y ajouter.

Flux	Format	Volume Bronze
patients (dump cumulatif)	CSV	18 000
sejours	CSV	6 797
diagnostics	JSON imbriqué	12 720
monitoring	Parquet	41 778
actes	Parquet	8 112
référentiels (services, CIM-10, description, CCAM)	CSV	36

Période : 1er au 29 août 2026. **Jeu synthétique** — les ordres de grandeur illustrent la chaîne, ils ne décrivent pas un CHU réel.

2. Architecture et choix



Couche	Rôle	Idempotence
Lake	Conformité RGPD uniquement	Fichier présent → skip
Bronze	Dépôt brut + <code>_source_date</code>	<code>meta.pipeline_runs</code> : date success jamais rechargée
Silver	Qualité, dédup, enrichissement	CREATE OR REPLACE TABLE
Gold	Règles métier, KPI, cloisonnement	CREATE OR REPLACE VIEW

Choix	Alternative écartée	Justification
ClickHouse	PostgreSQL	Moteur colonne : une agrégation ne lit que la colonne concernée, compression efficace sur des codes répétitifs. C'est le profil exact de la charge.
SQL dans le moteur	pandas	Sortir 41 000 relevés en mémoire Python ne passe pas à l'échelle — l'anti-pattern classique. Python se limite à copier et à envoyer les requêtes.
HMAC-SHA256 salé	SHA-256 simple	Sans sel, une attaque par dictionnaire sur la liste des IPP retrouve tous les pseudonymes. Le déterminisme préserve les jointures.
Vues Gold	Tables matérialisées	Aucune duplication, fraîcheur permanente, recréation instantanée.
Moteurs différenciés	ReplacingMergeTree partout	Déduplication seulement là où des doublons existent (patients, référentiels) : elle coûte du CPU pour rien ailleurs.
Parquet via Arrow	Listes Python	Protocole binaire natif, sans conversion intermédiaire, sur les deux flux les plus volumineux.
Règles métier en Gold	Règles métier en Silver	Silver répond « cette valeur est-elle plausible ? » (ingénierie), Gold « mérite-t-elle une alerte ? » (médical). Deux questions, deux responsables.

Incrémentalité concentrée en un point. Bronze seul est incrémental ; Silver et Gold sont reconstruits à chaque run (`CREATE OR REPLACE`, atomique, < 1 s). Aucun résidu d'un run raté ne survit, et une correction de règle se propage à tout l'historique. On évite ainsi la classe de bugs la plus pénible d'un entrepôt : deux couches qui divergent parce que l'une a été rejouée et pas l'autre.

Découverte des dates par union. Les flux ne couvrent pas la même période (`patients` 3 jours, activité 28, actes 1). Le pipeline prend l'union, un fichier manquant n'étant pas une erreur — sans quoi 25 jours disparaîtraient silencieusement.

3. Qualité des données

Bronze **n'écarte rien** : filtrer là ferait disparaître la preuve de l'anomalie et empêcherait tout audit. Silver applique les contrôles.

Contrôle	Traitement
Patients en dump cumulatif	dédupliqué par <code>argMax(_source_date)</code> → 6 000 patients
Sortie ≤ admission	écarté (68 séjours)
Monitoring hors plage physiologique	écarté (858 relevés)
Diagnostics / actes d'un séjour invalide	écartés (82 actes)
Sexe hors M/F, pseudo vide	écarté
Chapitre CIM-10	enrichi depuis la 1re lettre

`argMax` plutôt que `FINAL` : `FINAL` déduplique sans garantir quelle version survit, `argMax(valeur, _source_date)` retient délibérément la photo la plus récente — le comportement correct pour un dump cumulatif.

Résultat : 6 729 séjours, 40 920 relevés, 12 593 diagnostics, 8 030 actes.

4. RGPD et cloisonnement

Principe	Mise en œuvre
Pseudonymisation	HMAC-SHA256 salé dès le lake ; nom, prénom, NIR, <code>patient_id</code> supprimés ; date de naissance réduite à l'année
Minimisation	Pas de <code>region_code</code> ni <code>service_code</code> côté recherche ; date généralisée au mois
Cloisonnement	2 bases, 2 comptes ClickHouse, <code>SELECT</code> sur sa seule base
Petits effectifs	Seuil $k \geq 5$ patients distincts
Traçabilité	<code>_source_date</code> par ligne, statut et volume par run

Pourquoi dès le lake. Aucune donnée identifiante n'entre jamais en base — ni en Bronze, ni dans les sauvegardes, ni dans les logs. La pseudonymiser en Silver imposerait de la purger ensuite : plus fragile et invérifiable.

SQL SECURITY DEFINER — la vue comme frontière. Les comptes Gold n'ont aucun droit sur Silver, mais les vues s'exécutent avec les droits de leur créateur. L'accès se fait **au travers** de la projection définie : les colonnes exclues sont inatteignables, pas masquées.

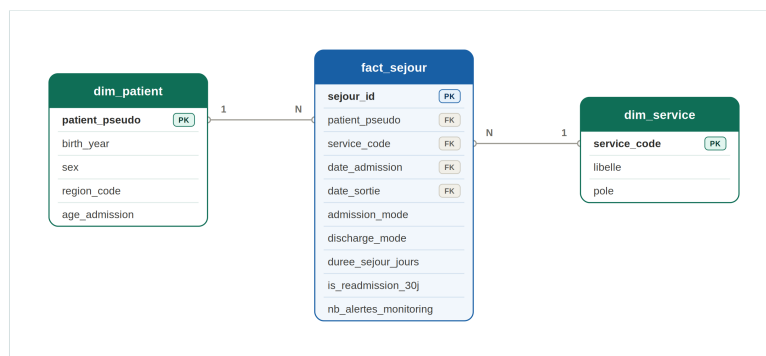
Le seuil $k \geq 5$ porte aussi sur la table de faits, pas seulement sur les vues agrégées — un `HAVING` posé sur les seuls KPI serait contournable en réagrégeant la fact brute.

Une fuite corrigée. La vue recherche exposait un `diagnostic_id` valant `concat(stay_id, '_', code_cim10, '_', type_diag)` : le `stay_id` prétendument exclu était récupérable par `splitByChar`, puis joignable sur le pilotage. Colonne retirée, inutile aux analyses de cohorte.

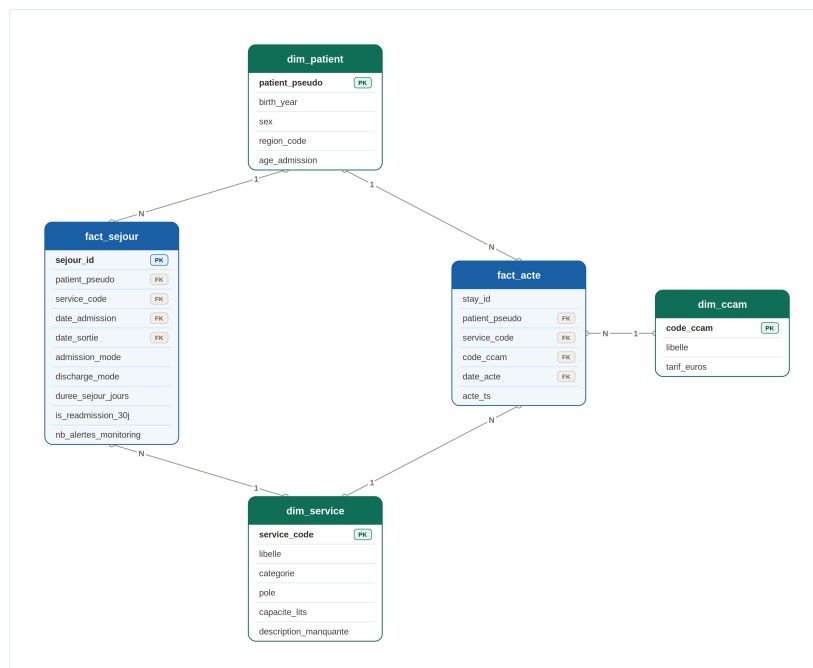
5. Modèle de données

Schéma en étoile plutôt que normalisé : moteur colonne, agrégations sans cascade de jointures, dashboards construits par des non-techniciens. Pas de `dim_temps` ni de flocon — les fonctions de date suffisent et les dimensions sont petites.

Avant l'évolution — grain : un séjour.

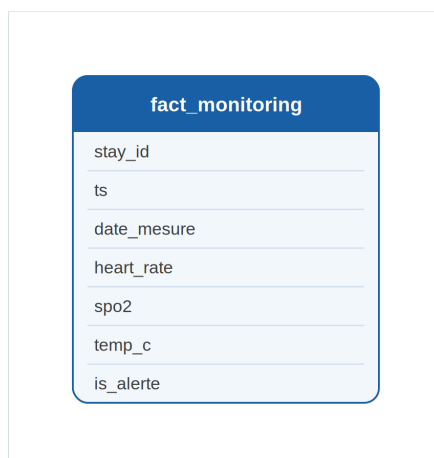


Après l'évolution — `dim_service` enrichie, deux nouveaux objets, `dim_service` désormais partagée par les deux facts.



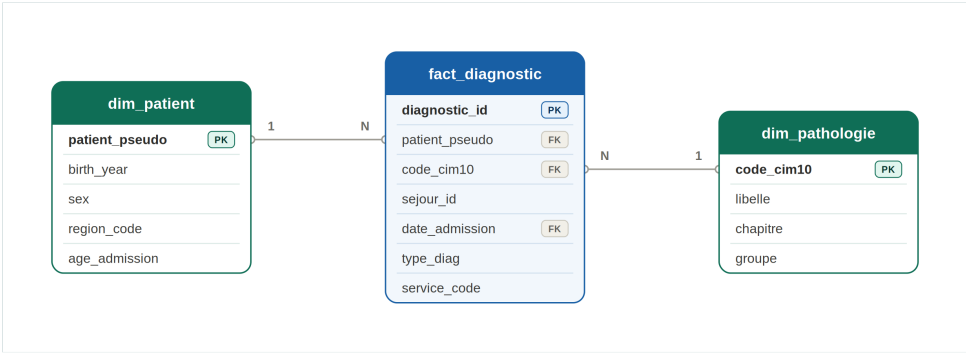
Le point à retenir du second schéma est **ce qu'il ne contient pas** : aucune liaison entre `fact_sejour` et `fact_acte`. Ce sont deux facts de grains différents ; les relier créerait un *fan trap* (cf. §7). `fact_acte` porte son propre `service_code`, dénormalisé à la construction, et ne joint que des dimensions.

Fait monitoring — grain : un relevé. Fact autonome, sans dimension propre : `ts` et `date_mesure` suffisent, les noms de capteurs sont fixes.



Le flag `is_alerte` est calculé en Gold, aux côtés de trois flags par type (désaturation, brady/tachycardie, fièvre) qui permettent de distinguer les natures d'alerte — Silver n'a fait que le contrôle de plausibilité.

Schéma recherche — inchangé par l'évolution. Grain : une occurrence de diagnostic, ce qui répond à « combien de patients portent le code I25 ? » par un simple `WHERE`, là où un grain séjour imposerait un tableau de codes.



Le diagramme montre le grain et les jointures ; la vue réellement exposée est **plus restrictive** — `diagnostic_id`, `sejour_id` et `service_code` en ont été retirés au titre du cloisonnement et de la minimisation (§4), et la date est ramenée au mois. Les actes n'y figurent pas : ils relèvent du pilotage et de la facturation, sans usage épidémiologique.

6. Les indicateurs

Le cahier liste quatre besoins de pilotage. Mais **une liste d'indicateurs n'est pas un tableau de bord** : ce qui en fait un outil, c'est qu'il répond à une chaîne de décision — **flux** → **stock** → **durée** → **intensité** → **valeur** → **qualité**. Les indicateurs ajoutés au cahier comblent les trous de cette chaîne.

Question	Indicateur	Pourquoi
Mes chiffres sont-ils à jour ?	Fraîcheur	Méta-indicateur : un dashboard figé qui n'affiche pas son âge est pire qu'absent
Qu'est-ce qui entre ?	Urgences / jour	Seule porte d'entrée non maîtrisée ; sa variation signale une crise avant tout autre
Qui est là ?	Séjours en cours	Le seul objet qui n'est pas un taux mais un fichier d'action
Combien de temps ?	DMS service / mois / catégorie	Arbitre rotation des lits contre qualité ; la catégorie est la maille d'arbitrage de direction
Avec quelle intensité ?	Actes/service, actes/type, densité/lit	L'occupation ne dit rien de la charge technique ; la densité rend comparables des services de tailles différentes
Qu'est-ce que ça rapporte ?	Montant T2A	Seul lien entre activité clinique et soutenabilité financière (le financement suit les actes)
Les soins sont-ils bons ?	Réadmission, mortalité, alertes	Qualité, et contrepoids aux indicateurs d'activité

Une lecture par paires. DMS et réadmission ne se lisent jamais séparément : une DMS qui baisse est une bonne nouvelle *sauf* si les réadmissions montent — signe de sorties prématurées. Le premier indicateur est manipulable au détriment du patient, le second détecte cette dérive.

Côté **recherche**, les quatre vues suivent la séquence de conception d'une étude : ai-je assez de patients, ma cohorte est-elle stable, qui sont-ils, qu'ont-ils d'autre — les comorbidités étant les facteurs de confusion sans lesquels aucune cohorte ne s'interprète.

Deux résultats notables. **La mortalité était un angle mort** : les décès n'existaient que dans la vue urgences, soit 681 sur 1 996 — **1 315 décès (66 %)** en séjour programmé ou mutation, invisibles du pilotage. **Le volume et la valeur s'inversent** : la radiographie du thorax est l'acte le plus fréquent et le moins cher (25 €, 25 950 € au total), l'appendicectomie le moins fréquent et pèse **trente fois plus** (800 €, 773 600 €).

7. Évolution : actes et hiérarchie de services

Le dépôt du 29 août ajoute un flux d'actes et un référentiel de description des services. Absorbé **en incrémental** — seule cette date a été traitée, les 28 précédentes n'ont pas été rejouées. Le delta de modèle est celui des deux schémas du §5 ; s'y ajoutent 8 030 actes et 5 vues KPI.

Piège 1 — référentiel incomplet. NEURO n'est pas décrit : 1 208 séjours, 18 % de l'activité. Un `INNER JOIN` l'aurait fait disparaître silencieusement de tous les KPI par catégorie. Choix : `LEFT JOIN`, **repli sur les étiquettes** (non `renseigne`) mais **NULL conservé sur la mesure**. Surtout pas `coalesce(capacite_lits, 0)` : zéro lit donnerait une division par zéro, ou une densité aberrante ressemblant à un vrai chiffre. Règle appliquée — *on remplit les étiquettes, jamais les mesures*. La ligne `non renseigne` visible devient un signal de qualité qui pousse le CHU à compléter son référentiel.

Piège de typage : `capacite_lits` étant `UInt16` non nullable, un `LEFT JOIN` sans correspondance la remplit avec le défaut du type — `0`, la valeur dangereuse. Tester l'échec de jointure était nécessaire pour forcer un vrai `NULL`.

Piège 2 — le service appartient au séjour, pas à l'acte. Joindre `fact_acte` à `fact_sejour` créerait un *fan trap* : grains différents, chaque séjour dupliqué autant de fois qu'il a d'actes, toute mesure du séjour multipliée en silence. Choix : **dénormaliser `service_code` dans `fact_acte`** à la construction, la `fact` ne joignant plus que des dimensions. C'est déjà ce que fait `fact_diagnostic`.

La hiérarchie n'est pas un arbre. `medecine` couvre deux pôles (Coeur-Poumon, Cancerologie) : catégorie et pôle sont deux regroupements **indépendants**, pas deux étages. Grouper sur le couple scinderait `medecine` et fausserait la lecture.

Non-régression : aucune vue existante redéfinie, `dim_service` modifiée de façon strictement additive (colonnes d'origine identiques, 8 lignes inchangées). Détail : `non-regression-evolution.md`.

8. Automatisation

Un service `pipeline` dans la stack Docker rejoue `python -m pipeline.run` selon un cron (`CRON_SCHEDULE, 0 6 * * *`), avec un `run` au démarrage.

Choix	Justification
Ordonnanceur dans le conteneur	L'hôte n'a besoin que de Docker : reproductible sur n'importe quelle machine, donc rejouable par un correcteur
supercronic plutôt que cron	Le cron classique exige un démon, n'écrit pas sur stdout (donc <code>docker compose logs</code> serait vide) et tronque l'environnement
Attente active de ClickHouse	<code>depends_on: service_healthy</code> garantit le conteneur sain, pas la base prête. 30 tentatives espacées de 2 s
Verrou <code>fcntl.flock</code>	Empêche cron et lancement manuel de se marcher dessus
Source montée <code>read_only</code>	Matérialise dans l'infra la contrainte du sujet : le CHU dépose, l'entrepôt lit

Garanties. Rejouer ne duplique rien (date `success` ignorée, Silver/Gold reconstruits). Une interruption ne laisse rien à moitié : fichiers écrits en `.tmp` puis renommés, et purge des lignes de la date avant réinsertion. Ce dernier point corrige un défaut réel — un crash entre deux des cinq insertions laissait des lignes partielles que le run suivant dupliquait. Vérifié en simulant une panne : les comptages retrouvent leurs valeurs de référence.

9. Gestion des erreurs

Le pipeline tourne sans surveillance : une erreur doit être **diagnosticable depuis les seuls logs**, sans rejouer le run en mode debug.

Principe	Mise en œuvre
Séparer l'attendu de l'inattendu	<code>PipelineError</code> couvre config et environnement (source absente, lac non créable, base injoignable) : message clair, sans traceback. Tout le reste remonte avec sa pile. L'exploitant distingue « j'ai mal configuré » de « il y a un bug ».
Échouer tôt sur le vérifiable	<code>SOURCE_DIR</code> existe, <code>LAKE_DIR</code> créable, <code>CLICKHOUSE_PORT</code> entier, <code>CRON_SCHEDULE</code> validé par <code>supercronic -test</code> . Une planification mal écrite fait échouer le conteneur au démarrage plutôt que de ne jamais se déclencher — panne silencieuse qu'on ne découvrirait que devant un dashboard figé.
Nommer le point de rupture	ClickHouse dit qu'une instruction a échoué, pas laquelle : <code>exec_sql_file</code> ajoute le fichier, le rang (i / N) et un extrait. Côté lecture, l'erreur porte fichier, date et nature — <code>patients.csv (2026-08-14)</code> : colonne manquante <code>'region_code'</code> se corrige sans ouvrir le code.
Ne rien laisser derrière	L'écriture atomique supprime le <code>.tmp</code> en cas d'échec : ni fichier tronqué pris pour « déjà fait », ni temporaire orphelin. Client fermé dans un <code>finally</code> , l'échec de fermeture étant absorbé pour ne pas masquer l'erreur d'origine.
Tolérer le tolérable	Dossier au nom non-ISO ignoré avec avertissement ; flux absent pour une date : pas une erreur ; échec du run de démarrage : le conteneur survit et le cron reprend au tick suivant.
Tracer et retenter	Statut <code>error</code> et message en base. Seules les dates en <code>success</code> étant sautées, la date échouée est retentée au run suivant, après purge de ses lignes partielles. Le retry découle du modèle, ce n'est pas un mécanisme séparé.

Deux comportements assumés : la boucle s'arrête à la première date en échec — les suivantes attendront le prochain run — et il n'existe pas de retry *au sein* d'un run.

10. Limites et recommandations

Le jeu est synthétique : actes répartis uniformément (variation 2,4 %), montant moyen par séjour quasi constant (415–451 €), mortalité globale à 16 %. Ces chiffres démontrent que la chaîne calcule juste, pas que l'hôpital va bien. S'y ajoutent un monitoring limité à 2 services sur 8, un seuil $k \geq 5$ n'écartant que 2 pathologies sur 13, et un mois d'historique — la hausse des DMS en septembre est un biais de composition, pas une tendance.

Limite	Recommandation
Traçabilité limitée à Bronze — <code>meta.pipeline_runs</code> déclare <code>layer</code> $\in$ {bronze, silver, gold}, seul <code>bronze</code> est écrit. Si Silver échoue, <code>v_data_freshness</code> affiche un pipeline sain.	Étendre <code>_record_run</code> à la couche et envelopper les trois <code>build_*</code> (~20 lignes)
Aucun retry en cours de run	Apport d'un orchestrateur (Prefect, Dagster), écarté ici : pipeline séquentiel, mono-machine, disposant déjà de son journal — arbitrer un graphe complexe n'aurait pas eu matière à s'exercer. L'arbitrage s'inverserait avec plusieurs flux concurrents.
Aucune alerte sur échec	Notifier au-delà d'un seuil d'ancienneté
Sel sans rotation ; montée en charge non éprouvée	Partitionner <code>monitoring</code> et <code>actes</code> par <code>_source_date</code> : la purge devient une opération de métadonnées
Anomalies monitoring = pannes de capteur — FC et SpO2 toujours hors plage ensemble, valeurs sentinelles, température jamais anormale	À valider au biomédical ; si confirmé, distinguer les sentinelles dès le Bronze et les remonter comme indicateur de maintenance, non de soin

Chiffres mesurés sur les dépôts du 1er au 29 août 2026. Analyse par indicateur et captures : [annexe-detaillée.pdf](#).